

Re-architecture of the DisloCluster 0-D Solver and of the 0-D $\leftrightarrow$ 3-D Coupling

DisloCluster development note

8 August 2026

Abstract

Three changes were made to the DisloCluster 0-D side and to its coupling with the spatially resolved code. (i) The right-hand side was moved into a single scalar-type-templated core so that the residual and an *exact* Jacobian, obtained by forward-mode automatic differentiation, are two instantiations of one definition; SUNDIALS' difference-quotient Jacobian is thereby retired. (ii) The six point-defect conservation accumulators were recognized as pure quadratures and moved out of the implicit system into CVODES quadrature variables, shrinking the Newton block from 19×19 to 13×13 standalone and to 9×9 under the operator split. (iii) The 0-D $\rightarrow$ 3-D handoff, which previously collapsed the marched state to four scalars, was replaced by a genuine per-node field exchange through MoDELib3's own cluster-dynamics configuration block.

Measurement then redirected the effort: at $n = 19$ the dense linear algebra was never the bottleneck. Profiling with the integration window collapsed to zero showed that per-case construction and destruction of SUNDIALS objects consumed 1.31 s of thread time against 0.2 s of actual solving in a 256-point coupling substep. Reusing one solver workspace per OpenMP thread cut the end-to-end march wall time by a factor of 1.67 at $q = 1024$ quadrature points — more than the two originally requested changes combined. The report states all three outcomes as measured, including the two that did not pay off in wall time.

Two further sections document the coupling in operation. Section 6.6 measures the full 1–26 dpa march on the 1 μm single crystal in three configurations, separating the genuine solver speedup ($1.95\times$ at identical work) from the cost of resolving a field on 24115 nodes instead of averaging 25 line points to four scalars. Section 7 characterizes MoDELib3's own nodal immobile integrator against the CVODE march — correcting, along the way, the common description of it as explicit Euler: it is a first-order semi-implicit scheme, and it has to be, because the measured stiffness puts its step 46 000 times beyond the forward-Euler limit. Section 8 then traces where each scheme came from, against the correct upstream: `mlm335/MoDELib-fullCD`, which already carries a full immobile field and its own IMEX integrator. MoDELib3's immobile solver is therefore a re-discretization of an existing scheme — nodal collocation and a single fused semi-implicit update, in place of fullCD's Galerkin mass-matrix projection followed by explicit Euler and sequential implicit loss factors — and not, as an earlier revision of this report stated on the strength of the wrong repository, an invention from nothing.

A later revision corrects the coupled route itself. As first written it did not perform an operator split: it read its mobile field out of a previously recorded standalone run, so $\mathbf{C}_M$ never responded to the immobile state the march was building. Section 10.1 puts MoDELib3's own steady mobile solve back inside the loop, through a new `useImmobileSolver` switch that lets a DDomp run perform the fast step alone. Section 10.3 then measures what that repairs: a 0-D seed at 0.1 dpa carries a C_v some $690\times$ above the quasi-steady value and no boundary layer at all, and the fast step supplies both corrections on its first call.

Contents

1 Starting point

3

2	Structural observation: the accumulators are pure quadratures	3
3	Implementation	4
3.1	One templated core, two instantiations	4
3.2	Reduced implicit block	4
3.3	Reusable per-thread solver workspace	4
3.4	Interface	5
4	Verification	5
4.1	The AD Jacobian	5
4.2	Bit-identity of the untouched path	6
5	Results	6
5.1	Standalone 0-D run	6
5.2	Operator-split coupling march	6
5.3	Where the improvement actually came from	7
5.4	A step-size pathology the reduction cures	7
5.5	The quadrature memo, and why the reduction does not win on time	8
6	The 0-D $\leftrightarrow$ 3-D field handoff	8
6.1	What was wrong	8
6.2	What MoDELib3 actually offers	8
6.3	The mapping	9
6.4	Verification on the 3-D verification case	9
6.5	Notebook wiring	10
6.6	Measured cost of the field upgrade	10
7	Integrator comparison: MoDELib’s nodal update versus the CVODE march	11
7.1	What MoDELib actually does	11
7.2	Test method	11
7.3	Stability	12
7.4	Accuracy and speed	12
7.5	Consequence for the coupling	13
8	Provenance: how the two schemes came to be	14
8.1	The upstream scheme, and how MoDELib3 re-discretized it	14
8.2	How MoDELib3’s variant reached its present form	16
8.3	The coupled march scheme: outward from the 0-D solver	16
8.4	Three routes to the same conclusion	17
9	The three recommendations, implemented and measured	17
9.1	Recommendation 1: how the accumulators are carried	18
9.2	Recommendation 2: the base+delta case protocol	18
9.3	Recommendation 3: deduplicating points before dispatch	19
9.4	Combined effect	19
10	The 1 μm reference through both routes	19
10.1	The fast step belongs inside the loop	20
10.2	A non-terminating mobile solve, and its cause	20
10.3	Does the split repair a seed that is not quasi-steady?	22
10.4	What this does and does not isolate	22
10.5	Speed	23
10.6	Attribution: model, splitting, or integrator?	23

10.7 A discrepancy with the calibration record	24
10.8 Output	24
11 Summary and recommended next steps	24

1 Starting point

The 0-D model integrates $N_{\text{eq}} = 19$ ordinary differential equations: 12 physical species, 6 point-defect conservation accumulators, and the evolving network dislocation density ρ_N . In the operator-split coupling the four mobile species $\mathbf{C}_M = [C_v, C_i, C_{2i}, C_{3i}]$ are frozen at the value supplied by the 3-D fast solve (`freeze_mobile=1`), and the remaining state is marched independently at every quadrature point, one case per point in an OpenMP batch subprocess.

Before this work the solver had the following properties.

- **No Jacobian was supplied.** `CVodeSetJacFn` was never called, so SUNDIALS formed the 19×19 dense Jacobian by difference quotients, costing 19 extra residual evaluations per Jacobian setup.
- **All 19 equations were implicit.** The accumulators were carried as ODE state, entered the Newton system and the error test, and were factorized in every LU.
- **Every case built its own solver.** `integrate_one` created and destroyed a `SUNContext`, state vector, dense matrix, linear solver and `CVODE` memory per invocation — i.e. once per quadrature point per substep.
- **The 0-D $\rightarrow$ 3-D carrier was four scalars.** The driver averaged the marched state over all points and wrote one number density and one mean radius per loop family into `MoDELib`'s microstructure generator inputs.

2 Structural observation: the accumulators are pure quadratures

Rows 12–17 of the right-hand side are written but columns 12–17 are never read: no equation in the system depends on an accumulator value. Inspection of `rate_equations.cpp` confirms that `y[12..17]` appears nowhere on the right of any assignment. The Jacobian therefore has the block structure

$$J = \begin{pmatrix} J_{ss} & 0 \\ J_{as} & 0 \end{pmatrix}, \quad s = \{\text{species}, \rho_N\}, \quad a = \{\text{accumulators}\}, \quad (1)$$

so the accumulators are quadratures of the state trajectory, $\mathbf{a}(t) = \int \mathbf{g}(\mathbf{y}_s(\tau)) d\tau$, and belong in `CVODES`' quadrature module rather than in the Newton system.

Combining this with the operator split, which freezes the four mobile species, gives the implicit block dimensions of Table 1.

Table 1: Dimension of the implicit block actually factorized.

Configuration	Newton block n	Dense LU $\propto n^3$	Relative LU cost
Legacy (all 19 implicit)	19	6859	1.00
Reduced, standalone	13	2197	0.32
Reduced, <code>freeze_mobile=1</code>	9	729	0.11

3 Implementation

3.1 One templated core, two instantiations

The physics was moved verbatim from `rate_equations.cpp` into `rate_equations_core.h` as

```
template <class T> void rhs_core(const T* y, T* ydot, const Parameters& P)
```

instantiated with `T = double` for the residual and with `T = Dual<N>` (`dual.h`) for the Jacobian. Forward-mode automatic differentiation evaluates value and all N directional derivatives in a *single* sweep: `sqrt`, `exp` and `cbrt` are evaluated once and their derivatives propagated by a multiply, rather than the N complete residual evaluations a difference quotient needs.

Two properties motivated AD over a hand-written analytic Jacobian. First, exactness: there is no perturbation size to choose and no truncation error. Second, and more important for maintenance, the Jacobian cannot drift out of sync with the residual — hand-differentiating the coalescence Avrami gates and the smoothstep minimum-size gate would have created a second copy of the physics to keep aligned, on a code base that already maintains a Python mirror.

Bit-identity. Floating-point addition is not associative, so the transcription preserved the exact grouping of every expression. Where the original wrote `k1 * k2 * C * D`, meaning $((k_1 k_2)C)D$, the templated core hoists $k_1 k_2$ into a named `double` and keeps the remaining order. This was verified rather than asserted: a reference binary was built from the git-HEAD sources and the outputs compared byte for byte (Section 5).

3.2 Reduced implicit block

With `-reduced=1` the state vector holds only the implicit components and the accumulators are advanced by `CVodeQuadInit`. Quadrature error control is left at its default (off), so the accumulators ride the step sequence the state equations demand instead of constraining it. The 19-column output contract is unchanged: each row is reassembled from the reduced state, the frozen mobile values and the quadrature vector.

3.3 Reusable per-thread solver workspace

This change was not planned; it was the result of profiling. Running a 256-case frozen-mobile batch with the integration window collapsed to $t_{\text{end}} = t_{\text{begin}}(1 + 10^{-12})$ — so that only process spawn, case-file parsing and output formatting remain — gave the decomposition in Table 2.

Table 2: Cost decomposition of one 256-case coupling substep, before the workspace change. “cpu-solve” is the sum of per-case integration wall times over all threads.

Configuration	wall [s]	cpu-solve [s]
Null integration (setup/teardown only)	0.172	1.310
1 dpa, 19×19 difference-quotient	0.136	1.608
1 dpa, 19×19 AD	0.129	1.491

Constructing and destroying the SUNDIALS objects cost 1.31 s of thread time against roughly 0.2 s of genuine integration: the per-case setup was consuming about six times what the ODE solve did. A `Workspace` is now created once per OpenMP thread and reused across every case that thread handles via `CVodeReInit` / `CVodeQuadReInit`, rebuilt only when a case needs a different shape. In a 1024-point march this reduces 1024 solver constructions per substep to 24, one per thread.

3.4 Interface

Three flags were added; both numerical flags default to *off* so that a standalone 0-D run remains bit-identical to the previous solver. `modelib_coupling.build_immobile_cases` turns them on for the march, where they are measurably better.

Flag	Effect
<code>-reduced=1</code>	Accumulators as CVODES quadrature; Newton block 13 (or 9 with <code>freeze_mobile</code>). CVODE only.
<code>-analytic_jac=1</code>	Exact dense Jacobian by forward AD. Dense linear solver only.
<code>-stats=1</code>	Emit a <code># STATS</code> comment line with the integrator counters. Non-numeric, so the existing Python row parsers skip it.

4 Verification

4.1 The AD Jacobian

A standalone tool (`jac_check.cpp`) compares the AD Jacobian against a central-difference Jacobian of the *same* core, differencing column j with a relative step $h_j = \varepsilon_{\text{rel}}|y_j|$ because the state spans some 34 decades. It was run at 20 states sampled along a converged trajectory from 10^{-8} to 100 dpa.

Two classes of entry are excluded, for reasons belonging to the finite difference and not to the AD:

- **FD-unresolvable entries.** A central difference can only see an entry whose contribution $J_{ij}h_j$ rises above the floating-point granularity of f_i . At the initial condition, dC_i/dt is pinned near $G_i \approx 10^{-7}$ while its C_v derivative moves it by $\sim 10^{-28}$, i.e. 10^{-5} of one unit in the last place. The difference quotient returns exactly zero there at *any* step size; AD returns the exact value. Entries are compared only where the signal exceeds 100 ULP. Note that this is precisely the blindness SUNDIALS' difference-quotient Jacobian suffers, so the AD Jacobian is strictly more informative in this region.
- **Components on the concentration floor,** where `f1()` makes the residual non-smooth: AD returns the one-sided derivative while a central difference straddles the kink.

The result is in Table 3. The discrepancy falls by two decades per decade of ε_{rel} across the usable range, the $O(h^2)$ signature of a correct derivative with the finite difference's own truncation error dominating.

Table 3: AD versus central-difference Jacobian, worst row-relative discrepancy over all compared entries. Representative states from the 20 tested.

Dose [dpa]	$\varepsilon_{\text{rel}} = 10^{-5}$	10^{-6}	10^{-7}	entries compared
1.0×10^{-8}	6.50×10^{-12}	8.84×10^{-11}	1.24×10^{-9}	6
1.3×10^{-6}	8.49×10^{-8}	7.96×10^{-7}	2.05×10^{-6}	108
1.6×10^{-4}	8.29×10^{-8}	9.97×10^{-7}	1.83×10^{-5}	117
2.1×10^{-2}	4.40×10^{-7}	4.71×10^{-6}	5.16×10^{-5}	133
7.8×10^{-1}	1.55×10^{-7}	9.30×10^{-7}	9.92×10^{-6}	133
8.9×10^0	3.09×10^{-9}	4.28×10^{-8}	1.30×10^{-6}	107
1.0×10^2	8.04×10^{-9}	5.17×10^{-8}	5.71×10^{-7}	107
Worst over all 20 states (best ε per state): 4.40×10^{-7} ; states failing at every ε : 0/20.				

4.2 Bit-identity of the untouched path

The legacy configuration of the rebuilt solver reproduces the git-HEAD reference binary *byte for byte* — 1000 output rows $\times$ 19 columns in the standalone run, and the endpoint state of all 256 points $\times$ 5 substeps in the coupling march. The refactor into a templated core therefore changed no arithmetic.

5 Results

All runs use `input/Zr_input_parameters.xlsx` at $T = 573$ K, $G = 10^{-7}$ dpa/s, on 24 cores. Because wall-clock times on a shared machine are noisy, exact operation counts are reported alongside and are the primary evidence:

$$\begin{aligned} n_{\text{core}} &= \text{residual-core evaluations} = \text{nfe} + \text{nfeLS} + \text{nqmiss}, \\ n_{\text{LU}} &= \text{dense factorizations (nsetups)}, \quad \text{LU flops} \propto n_{\text{LU}} n^3/3. \end{aligned}$$

5.1 Standalone 0-D run

$t = 10^{-1} \dots 10^9$ s (100 dpa), 1000 output points, $\text{rtol} = 10^{-6}$, $\text{atol} = 10^{-20}$. Accuracy is measured against a tight reference ($\text{rtol} = 10^{-10}$, $\text{atol} = 10^{-24}$), compared per column only where the reference exceeds 10^{-8} of that column’s own peak — a plain elementwise relative error is meaningless on a state spanning 34 decades whose early transient sits below atol .

Table 4: Standalone 0-D run. “bit=ref” is byte-for-byte agreement with the git-HEAD binary.

Configuration	wall [ms]	solve [ms]	nst	n_{core}	n_{LU}	LU flops	bit=ref
git HEAD 19 $\times$ 19 DQ	70.7	—	—	—	—	—	—
Baseline 19 $\times$ 19 DQ	69.3	46.7	2797	6129	466	1.07×10^6	yes
AD Jac. 19 $\times$ 19 AD	65.5	46.9	2483	4171	429	9.81×10^5	no
Reduced 13 $\times$ 13 DQ	64.5	42.3	2564	7823	423	3.10×10^5	no
Reduced + AD 13 $\times$ 13	63.4	43.7	2811	7415	458	3.35×10^5	no

Table 5: Standalone accuracy against the tight reference (max relative error).

Configuration	species	accumulators	ρ_N
git HEAD / baseline 19 $\times$ 19 DQ	1.41×10^{-4}	2.15×10^{-5}	1.60×10^{-5}
AD Jac. 19 $\times$ 19	2.18×10^{-4}	4.90×10^{-5}	3.16×10^{-5}
Reduced 13 $\times$ 13 DQ	2.44×10^{-4}	5.25×10^{-5}	3.30×10^{-5}
Reduced + AD 13 $\times$ 13	8.79×10^{-5}	2.18×10^{-5}	1.30×10^{-5}

The AD Jacobian removes 32% of the residual evaluations ($6129 \rightarrow 4171$) and 11% of the steps. The reduction cuts LU work by 71% but *adds* residual evaluations, for the reason given in Section 5.5. All four configurations sit within a factor of three of each other in accuracy, at the level expected from different step sequences at the same tolerance; none is meaningfully more accurate than another.

5.2 Operator-split coupling march

A 5 dpa dose step in 5 substeps, seeded at 1 dpa, with the mobile field depleted toward a sink across the point set. Table 6 gives $q = 1024$; the $q = 64$ and $q = 256$ rows scale proportionally.

Table 6: Coupling march, $q = 1024$ quadrature points, 5 substeps. “warn” counts CVODE roundoff warnings (see Section 5.4); error is against the git-HEAD result.

Configuration	wall [s]	cpu-solve [s]	nst	n_{core}	n_{LU}	LU flops	warn
git HEAD 19×19 DQ	2.379	—	—	—	—	—	2048
Reuse 19×19 DQ	1.424	12.03	175771	366170	69860	1.60×10^8	2048
Reuse + AD 19×19	1.435	12.98	175890	243682	69921	1.60×10^8	2048
Reduced 9×9 DQ	1.568	13.99	143651	400404	54959	1.34×10^7	0
Reduced + AD 9×9	1.572	13.68	143654	349626	54957	1.34×10^7	0

Table 7: March wall time versus point count. Speedup is against the git-HEAD binary at the same q .

Configuration	$q = 64$		$q = 256$		$q = 1024$	
	[s]	speedup	[s]	speedup	[s]	speedup
git HEAD	0.254	1.00	0.692	1.00	2.379	1.00
Reuse 19×19 DQ	0.214	1.19	0.482	1.44	1.424	1.67
Reuse + AD 19×19	0.211	1.20	0.468	1.48	1.435	1.66
Reduced + AD 9×9	0.217	1.17	0.502	1.38	1.572	1.51

Agreement with the git-HEAD march is 1.3×10^{-5} or better in every species at every q — the expected splitting-consistent difference between step sequences, not a change of answer.

5.3 Where the improvement actually came from

Stated plainly, and against the expectation that motivated the work:

- **Workspace reuse is the whole wall-clock gain:** $1.67\times$ at $q = 1024$, and it grows with q because the per-case setup it removes scales with the point count while the per-thread construction does not.
- **The AD Jacobian delivers its target metric:** residual evaluations fall by 33% ($366\,170 \rightarrow 243\,682$) and difference-quotient evaluations go to zero. Wall time is neutral, because one Dual<19> sweep costs about what the 19 scalar evaluations it replaces cost. Its durable value is exactness, the elimination of the FD-blind entries of Table 3, and a Jacobian that cannot fall out of step with the residual.
- **The 9×9 reduction delivers its target metric — a 92% cut in LU flops — but LU was never the bottleneck at $n = 19$.** It also reduces the step count by 18%. It nonetheless loses slightly in wall time, because of the quadrature cost analyzed next.

5.4 A step-size pathology the reduction cures

The warn column of Table 6 is not cosmetic. The legacy path emits, on every substep of every point,

[WARNING] Internal $t = 9.88525\text{e}+06$ and $h = 2.10129\text{e}-12$ are such that $t + h = t$ on the
next step

2048 of them in a 1024-point, 5-substep march.

The cause is the accumulators. Each substep restarts at $t \approx 10^7$ s with the accumulators reset to zero, and with $\text{atol} = 10^{-20}$ their error weights $w_i = 1/(\text{rtol}|y_i| + \text{atol})$ are $\sim 10^{20}$. CVODE’s initial-step heuristic uses the weighted norm of $\dot{\mathbf{y}}$; with $\dot{a} \sim 10^{-7}$ this gives $\|\dot{\mathbf{y}}\|_{\text{WRMS}} \sim 10^{13}$ and

hence $h_0 \sim 10^{-13}$, while $\varepsilon t \approx 2 \times 10^{-10}$. The first step is below the representable resolution of t and is wasted.

Removing the accumulators from the error test eliminates this outright: zero warnings, and 18% fewer internal steps. This is the reduction’s most defensible benefit — it is a robustness property, independent of problem size, whereas the LU saving is not.

A second, subtler effect works the other way. The error test is a root-mean square over components, $\|e\|_{\text{WRMS}} = \sqrt{N^{-1} \sum_i (e_i w_i)^2}$, so removing six well-behaved components makes the norm $\sqrt{19/9} = 1.45$ times more sensitive per remaining component. The 19-equation system was being flattered by six dummy components diluting its error norm; the reduced system is held to a genuinely tighter standard at the same nominal tolerance.

5.5 The quadrature memo, and why the reduction does not win on time

CVODES calls the quadrature right-hand side once per completed step. Because that call needs the same core evaluation the residual has just performed, the implementation memoizes the accumulator rates on the exact $(t, \mathbf{y})$ bit pattern and reuses them on a match.

The measured hit rate is poor: 1 hit against 2564 misses in the standalone run. CVODE’s corrector evaluates the residual at its Newton iterates, and the accepted step point $\mathbf{y}_n$ is generally not one of them, so the memo misses and a second full core sweep is paid per step. That is exactly the +28% in n_{core} visible in Table 4, and it cancels the LU saving.

The identified remedy, not implemented here, is to keep the accumulators in the state vector but neutralize their error weights with a vector `atol` (`CvodeSVtolerances` with `atol = 10300` on those six components). That would retain the step-size cure of Section 5.4 at *zero* extra core evaluations, at the cost of a 15×15 rather than 9×9 block — which, on this evidence, is the better trade. It is recorded as the next step rather than claimed as a result.

6 The 0-D $\leftrightarrow$ 3-D field handoff

6.1 What was wrong

The previous 0-D $\rightarrow$ 3-D carrier (`modelib_fem.write_sink_field`) wrote four scalars per dose step — one number density and one mean radius per loop family — into `inputFiles/aLoopsDensity.txt` and `inputFiles/frankLoopsDensity.txt`. Those files drive MoDELib’s *microstructure generator*, which places discrete loops at a uniform target density. The driver therefore had to collapse the marched state to its volume average before handing it over:

```
A = np.mean(np.asarray(Y), axis=0)
```

Every bit of spatial structure the slow march produced was discarded on the way out. In the reverse direction the mobile field was read from `F/quadrature_*.txt` through a column map whose source comment conceded it had never been checked against a real run.

6.2 What MoDELib3 actually offers

MoDELib3 does not need a scalar. `ClusterDynamicsFEM` already carries the immobile population as a finite-element *field* — `immobileClusters`, with `iSize = 8` components per node — and `ImmobileSinks` builds the sink strength from the local value at every quadrature point. `ClusterDynamics::output` writes that field, together with the `mSize = 4` mobile components, into the CD block of `ev1/ev1_<N>.txt`. Decisively, `ClusterDynamicsFEM::initializeConfiguration` reads it back:

```
if(size_t(configIO.cdMatrix().size())==mobileClusters.gSize()+immobileClusters.gSize())
{
    mobileClusters = configIO.cdMatrix().block(0,0,nNodes,mSize)...
```

```

    immobileClusters = configIO.cdMatrix().block(0,mSize,nNodes,iSize)...
}

```

The CD block is therefore a read/write field channel in both directions, on the node set the two codes already share. `evl/cdNodes.txt` gives the node coordinates in the same row order, so no search or interpolation is required.

6.3 The mapping

Table 8: CD block layout, `ClusterDynamicsParameters`: `mSize = 4`, `iSize = 8`. `immobileSpeciesVector = -1 1 1 1` makes family 0 the basal $\langle c \rangle$ vacancy family and families 1–3 the three prism $\langle a \rangle$ variants.

Columns	Block	Components	Units
0–3	mobile	C_v, C_i, C_{2i}, C_{3i}	atom fraction
4–7	immobile	$n_c, n_{a1}, n_{a2}, n_{a3}$	per b^3
8–11	immobile	$c_c, c_{a1}, c_{a2}, c_{a3}$	atom fraction

MoDELib works in Burgers-vector units, so its nodal number density is per b^3 while the 0-D loop density is an atom fraction. `ImmobileSinks` floors them as $n_{\text{floor}} = C_{\text{floor}}/\omega$ and $c_{\text{floor}} = C_{\text{floor}}$, which fixes the conversion:

$$n_{\text{MoDELib}} = \frac{C_{0D}}{\omega}, \quad c_{\text{MoDELib}} = c_{0D}, \quad \omega = \frac{\text{atomicVolume_SI}}{b_SI^3} = 0.355111 \, b^3. \quad (2)$$

The 0-D aligned/non-aligned pair is collapsed and the $\langle a \rangle$ total distributed over the three prism variants by the resolved-stress weights — the same Option-A reduction `modelib_export` performs, but applied *pointwise* rather than to a volume average. The mobile direction is a pure slice: MoDELib’s four mobile species *are* the 0-D mobile species, in the same order and units.

6.4 Verification on the 3-D verification case

Tested against MoDELib3/tutorials/zrmicro_coupled, the 1 μm single-crystal case with 24 115 CD nodes, at `evl_30.txt`.

Table 9: Field-handoff verification, 24 115 CD nodes.

Check	Result
CD block rewrite round-trip	identical, $24\,115 \times 12$; header and all dislocation records preserved
0-D $\rightarrow$ MoDELib $\rightarrow$ 0-D immobile	max relative error 4.47×10^{-16} (machine precision)
Mobile field read back	C_v spans 8.46×10^{-15} to 6.70×10^{-6} , a dynamic range of 7.9×10^8
Spatial structure	$\text{corr}(\log_{10} C_v, \text{distance to nearest face}) = +0.63$
March on the real node set	24 115 nodes over 1 dpa in 9.5 s, 0 failures
Field written back	n_{a1} spans a factor 4.4 across the domain

The information content of the handoff rises from 4 numbers to $24\,115 \times 12 = 289\,380$ numbers, and the boundary-layer structure that the Dirichlet condition imposes on the mobile field — eight decades in C_v , correlating with distance to the surface — now reaches the slow march instead of being averaged away.

A precision detail. Restoring the aligned/non-aligned split on the return trip initially lost eight significant digits on the minor partner. The fractions were formed as f and $1 - f$; when the pair is lopsided (and $C_{iL} \gg C_{aiL}$ is normal at low dose) f rounds to $1 - \epsilon$ and $1 - f$ retains only the bits surviving the cancellation. Forming both fractions directly from the pair restores full precision, giving the 4.47×10^{-16} of Table 9.

6.5 Notebook wiring

`coupled_0d_3d_ZrMicro.ipynb` gains a "field" fast-solve backend, selected automatically when a CD field is present. In that mode the march runs on the CD node set rather than the synthetic 1-D line, seeds the immobile state from the 3-D field itself, and after each dose interval writes `evl/evl_coupled_<dose>dpa.txt` — a complete configuration, with the dislocation records copied through unchanged, that MoDELlib can be restarted from. The previous scalar path and the analytic placeholder both remain available and unmodified.

Because MoDELlib reads its configuration at start-up, the handback takes effect on a *restart* from the written file. That is the rhythm the operator split requires anyway (fast solve $\rightarrow$ slow march $\rightarrow$ fast solve), but it does mean the coupling is restart-driven rather than in-process: at the default six snapshots that is six DDomp launches, which is unremarkable; at hundreds of dose intervals it would argue for driving MoDELlib through `pyMoDELlib` instead.

6.6 Measured cost of the field upgrade

The comparison needs care, because the old path never ran on the mesh at all. `RUN["n_points"] = 25`: it marched a synthetic 1-D line of 25 points bearing no relation to the geometry, while the field path marches the 24 115 CD nodes — 965 times as many. A same-work control is therefore included: the git-HEAD solver driven over the *same* 24 115 nodes, which is what the old solver would have cost to do the new job.

Test. The notebook’s default schedule — 1 to 26 dpa, five intervals, `substeps_per_interval = 20`, i.e. 100 batch subprocesses — run to completion in all three configurations, seeded at 1 dpa from the 0-D reference and, for the field configurations, from the CD field itself.

Table 10: Full 1–26 dpa coupling march, 100 batch subprocesses, 24 cores.

	Configuration	q	wall [s]	s/substep	ms/point/substep
A	Old as configured (git HEAD, 1-D line)	25	4.3	0.04	1.715
B	Old solver at the new workload	24 115	1117.4	11.17	0.463
C	New: $9 \times 9 + \text{AD} + \text{reuse}$	24 115	574.2	5.74	0.238

Two distinct readings, and both are needed to avoid misrepresenting the change:

- **Same work, B versus C: 1.95 $\times$ faster.** That is the honest solver speedup — identical points, identical schedule. It exceeds the 1.67 $\times$ measured at $q = 1024$ in Table 7 because the per-case solver construction the old binary pays scales with q while the new binary’s per-thread construction does not: 24 115 solver builds per substep become 24.
- **Different work, A versus C: 134 $\times$ more wall clock for 965 $\times$ the points.** The old configuration is faster in absolute terms only because it was doing 1/965 of the work and emitting eight numbers. Per point per substep the new path is 7.2 $\times$ cheaper (1.715 $\rightarrow$ 0.238 ms), part solver improvement and part fixed process-and-parse overhead amortizing over far more cases.

For scale: the 3-D solve itself (`clean_run.sh` $\rightarrow$ DDomp) takes hours. Against that, neither 4 s nor 10 min makes the march the pacing item of a coupled run.

Consistency. Over the full 1–26 dpa march, configurations B and C agree to 9.05×10^{-7} in the species. That is *tighter* than the 8.2×10^{-4} measured after the first 5 dpa interval alone, and the direction is informative: the early transient difference between two step sequences washes out rather than accumulating, because the loop populations are approaching a quasi-steady state and the dynamics are attracting. The largest disagreements sit at the stiff near-boundary nodes, where the frozen mobile field is eight decades below the interior value. It is a tolerance effect, not a change of answer.

7 Integrator comparison: MoDELib’s nodal update versus the CVODE march

The two codes advance the *same* immobile physics — eight components per node — with entirely different schemes. This section characterizes both, since the difference is a systematic source of 0-D/3-D discrepancy that is independent of the physics non-correspondences already documented in `CLAUDE.md`.

Everything said here about “MoDELib” means MoDELib3, the standalone route actually used in this work. The upstream MoDELib-fullCD discretizes the same physics differently again — a Galerkin mass-matrix projection of the rate, then explicit Euler and sequential implicit loss factors, one step per dose step rather than twenty — and is characterized separately in Section 8.1.

7.1 What MoDELib actually does

It is worth stating precisely, because the scheme is often described as explicit Euler and it is not. `ClusterDynamicsFEM::solveImmobileClusters` loops over nodes and, inside each node, over `nSub = 20` substeps of $dt = dtMax/20$, applying

```
// implicit for the linear losses, explicit for the sources
const double nNew((n(k)+dt*nucRate)/(1.0+dt*lossN));
const double cNew((c(k)+dt*(gdot(k)+Gk+clusShare(k)*clusC(pol)-contentSink))/(1.0+dt*lossC));
```

This is a first-order **semi-implicit (IMEX)** update: the sources (nucleation, absorbed flux, cascade seed) are explicit, while the linear loss coefficients — coalescence, $1/\tau_{vL}$, and the thermal-emission rate recast as a first-order coefficient — sit in the denominator. The source comment records the reason: “ τ_{vL} is short compared with a dose step ... an explicit update would give $n(1 - dt/\tau) < 0$ and collapse the vacancy-loop density.”

With $G = 10^{-7}$ dpa/s and `dtMax = 6.95868964e19` in MoDELib units (time unit 1.4371×10^{-13} s), one dose step is 10^7 s and the immobile substep is

$$dt_{\text{MoDELib}} = \frac{10^7 \text{ s}}{20} = 5 \times 10^5 \text{ s.} \quad (3)$$

7.2 Test method

Three instruments, all exercising the *same* `rate_equations_core.h` right-hand side so that no model difference contaminates the comparison:

1. **Spectrum.** The frozen-mobile 9×9 immobile Jacobian is formed at states sampled along a converged trajectory and its eigenvalues taken. The forward-Euler stability limit is $dt < 2/|\text{Re } \lambda|_{\text{max}}$.
2. **Fixed-step explicit reference.** A `-euler_nsub=N` mode was added to the solver: N forward-Euler substeps per output interval on the same core, with the per-substep positivity clamp MoDELib applies. This is the *control* — what the update would do without the implicit loss treatment — not a model of MoDELib’s scheme.

3. **Reference solution.** CVODE at $\text{rtol}=1\text{e-}12$, $\text{atol}=1\text{e-}26$ with the AD Jacobian, against which every other result is measured.

Two nodes of the 1 μm case bracket the range: the interior node of maximum C_v and the near-boundary node of minimum C_v .

7.3 Stability

Table 11: Spectrum of the frozen-mobile 9×9 immobile Jacobian.

Dose [dpa]	$ \text{Re } \lambda _{\max} [\text{s}^{-1}]$	$dt_{\text{stab}} = 2/ \text{Re } \lambda _{\max}$	stiffness ratio	$dt_{\text{MoDELib}}/dt_{\text{stab}}$
0.99	1.854×10^{-1}	10.8 s	1.5×10^5	46 338
2.96	1.853×10^{-1}	10.8 s	5.7×10^7	46 335
5.91	1.853×10^{-1}	10.8 s	4.0×10^5	46 335
11.16	1.853×10^{-1}	10.8 s	1.2×10^5	46 335
15.78	1.853×10^{-1}	10.8 s	3.1×10^4	46 335
21.05	1.853×10^{-1}	10.8 s	1.8×10^7	46 335
26.52	1.853×10^{-1}	10.8 s	9.9×10^4	46 335

The fast eigenvalue is essentially dose-independent at 0.185 s^{-1} — the vacancy-loop dissolution channel, $n_{vL,\text{nuc}}/\tau_{vL}$ — so a forward-Euler step must satisfy $dt < 10.8 \text{ s}$. MoDELib runs at $5 \times 10^5 \text{ s}$, **46 000 times the explicit limit**. Its implicit loss treatment is therefore not a refinement; it is what makes the scheme usable at all.

Table 12 shows the consequence directly, from the fixed-step explicit control at MoDELib’s own substep count.

Table 12: One 1 dpa step at the near-boundary node: a genuine forward Euler at $dt = 5 \times 10^5 \text{ s}$ annihilates the entire $\langle c \rangle$ vacancy-loop family, exactly the failure the MoDELib source comment anticipates. The $\langle a \rangle$ families, which carry no fast decay channel, are unaffected.

Species	initial	CVODE reference	explicit Euler, $N = 20$	rel. error
C_{iL}	2.53×10^{-3}	5.69×10^{-3}	5.69×10^{-3}	1.8×10^{-11}
C_{aiL}	1.60×10^{-3}	3.44×10^{-3}	3.44×10^{-3}	2.9×10^{-11}
$C_{iL,i}$	2.16×10^{-1}	2.79×10^{-1}	2.79×10^{-1}	2.2×10^{-10}
$C_{aiL,i}$	1.37×10^{-1}	1.73×10^{-1}	1.73×10^{-1}	2.3×10^{-10}
C_{vL}	3.48×10^{-10}	3.48×10^{-10}	1.0×10^{-20} (floor)	1.00
C_{avL}	2.02×10^{-10}	2.02×10^{-10}	1.0×10^{-20} (floor)	1.00
$C_{vL,v}$	7.21×10^{-4}	7.21×10^{-4}	1.0×10^{-20} (floor)	1.00
$C_{avL,v}$	4.20×10^{-4}	4.20×10^{-4}	1.0×10^{-20} (floor)	1.00

Both schemes actually in use are unconditionally stable on the stiff channel, but by different mechanisms. MoDELib’s amplification factor $1/(1 + dt \lambda) \in (0, 1)$ for every $dt > 0$ and $\lambda > 0$: L-stable on the linear part, and positivity-preserving by construction. CVODE BDF is A-stable at orders 1–2 and stiffly stable to order 5, and in addition selects its own step from a local error estimate.

7.4 Accuracy and speed

At the near-boundary node CVODE needs only 37 steps and 84 core evaluations for 1.27×10^{-8} , the frozen mobile field there being effectively zero.

Table 13: One 1 dpa dose step, interior node ($C_v = 6.70 \times 10^{-6}$), against the `rtol=1e-12` reference. “core evals” counts evaluations of the shared right-hand side.

Integrator	substeps	dt [s]	core evals	max rel. error	t [ms]
Explicit Euler (MoDELib’s <code>nSub</code>)	20	5.0×10^5	20	9.27×10^4	0.14
Explicit Euler	200	5.0×10^4	200	9.27×10^3	0.27
Explicit Euler	2000	5.0×10^3	2000	9.26×10^2	1.18
Explicit Euler	20000	5.0×10^2	20000	9.17×10^1	13.71
CVODE BDF $9 \times 9 + \text{AD}$, <code>rtol=1e-6</code>	161 (adaptive)	6.2×10^4	366	2.10×10^{-6}	1.50

The explicit control converges cleanly at first order — a factor 10 in error per factor 10 in step count — which confirms the order; the enormous constant reflects that it is simultaneously unstable.

What can and cannot be claimed for MoDELib’s scheme. Its IMEX splitting was not reimplemented on the ZrMicro equations, so no error *number* is quoted for it here. What is structural, and visible in the code itself, is that it is **first-order accurate with no error control**: no local error estimate, no step rejection, no adaptivity, and 20 fixed substeps per dose step whatever the local stiffness. Against that, CVODE selects its own order (up to 5) and step from a local error estimate at `rtol` = 10^{-6} .

Speed. Per node per dose step, MoDELib performs 20 nodal updates of eight components with no Jacobian, no linear solve and no error estimate; the comparable explicit control costs 0.14 ms against CVODE’s 1.50 ms interior and 0.76 ms near the boundary. So MoDELib’s immobile update is roughly **an order of magnitude cheaper per node**, which is precisely the price of first order and no adaptivity. At the scale of the coupled march this is the difference between the 3-D solve’s nodal update being free and the 0-D march costing the 574 s of Table 10.

7.5 Consequence for the coupling

Table 14: The two integrators side by side.

	MoDELib <code>solveImmobileClusters</code>	ZrMicro coupled march
Scheme	semi-implicit (IMEX), nodal	CVODE BDF, adaptive
Order	1	up to 5
Step	fixed, 5×10^5 s (20/dpa)	adaptive (~ 161 /dpa interior)
Error control	none	local estimate, <code>rtol</code> = 10^{-6}
Stability	L-stable on the linear losses	stiffly stable to order 5
Linear algebra	none (diagonal by construction)	dense 9×9 LU, exact AD Jacobian
Cost per node	~ 0.14 ms / dpa	~ 0.8 – 1.5 ms / dpa
Accuracy	first order, unquantified	2×10^{-6} measured

The operator split is itself first order in $\Delta\gamma$ (Section 7 notwithstanding, this is the scheme’s design order). MoDELib’s immobile update is therefore *consistent with* the splitting order, and CVODE’s six additional digits on the slow march buy nothing unless $\Delta\gamma$ is refined alongside them.

Where the difference does matter is the return path. Once the marched field is written back through the CD block (Section 6.6), the two schemes’ different truncation errors become a systematic 0-D/3-D discrepancy — a *fourth* candidate alongside the three physics non-correspondences

listed in `CLAUDE.md` (the DAD bias reconciliation, the bi-pyramid vacancy family, and size-dependent vacancy-loop emission). Anyone chasing a residual disagreement between the interior 0-D state and the 3-D result should rule this one out first, because it is the cheapest to test: raise `nSub` in `solveImmobileClusters` and see whether the disagreement moves.

8 Provenance: how the two schemes came to be

Section 7 compared the two integrators as they stand. This section records where each came from, because the histories explain the asymmetry between them and carry a maintenance consequence.

Correction: the upstream is MoDELib-fullCD, not MoDELib2-NNL. An earlier revision of this section took MoDELib2-NNL as the ancestor of MoDELib3’s cluster dynamics and concluded, from that repository, that “the immobile field does not exist at all upstream”. Every factual claim made about that repository holds — a checkout confirms `iSize = 0`, an empty `solveImmobileClusters()` body, no `ImmobileSinkRate.h` or `SpatialODESolver.h`, and the literal `// Missing immobile sinks placeholder at ClusterDynamicsFEM.cpp:110` — but the baseline was the wrong one, so the conclusion drawn from it does not follow. The CD-complete upstream is [m1m335/MoDELib-fullCD](#) (commit 62ac34f), which carries a full immobile field, its own immobile integrator, and the loop sink on the mobile equation. Everything below has been re-derived from a checkout of that repository, and the conclusions change: the MoDELib3 immobile solver is a *re-discretization* of an existing upstream scheme, not an invention from nothing.

The lineage is checkable in the parameter set. Of the cluster-dynamics keys read by `ClusterDynamicsParameters` 31 are common to both codes verbatim, and most of the remainder are renames of one another:

MoDELib-fullCD	MoDELib3 (Zr3d_ghoniem)
<code>loopAnnealTau0_SI, loopAnnealEa_eV</code>	<code>tau0_vLoop_SI, Ea_vLoop_eV</code>
<code>loopNucDefects</code>	<code>nNuc</code>
<code>loopCoalLL, loopCoalLN</code>	<code>cLL, cLN</code>
<code>loopCoalKappaLL, loopCoalKappaLN</code>	<code>kappaLL, kappaLN</code>
<code>loopCoalNetwork_SI</code>	<code>rhoNetwork_SI</code>
<code>loopCascadeFraction</code>	<code>loopCascadeFractions</code>
<code>minimumLoopSize</code>	<code>r_min</code>
<code>initLoopSinks_SI</code>	<code>initloopSinks_SI</code>

What is genuinely *new* in MoDELib3 is small and specific: `dadAnisotropy` and `dadZ0` (the DAD reconciliation with the 0-D), `atomicVolume_SI` (the CD atomic volume decoupled from the lattice cell volume), `concentrationFloor` (ZrMicro’s C_{floor}) and `loopSinkScale`. What is genuinely *lost* is `clusterDiscretizationTime/clusterDiscretizationFactor` and with them `initializeDiscreteClimbLoops()`: `fullCD` converts its continuum loop field into discrete climb loops once the dose reaches a set threshold, and MoDELib3 has no such transition at all. That capability is what the name “fullCD” denotes, and dropping it is the largest functional regression of the fork.

8.1 The upstream scheme, and how MoDELib3 re-discretized it

`fullCD` advances the immobile field in three distinct stages per DD step.

1. **Rate, by Galerkin projection.** `ImmobileSinkRate` is an `EvalFunction` evaluated at *quadrature points* — absorption growth, cascade nucleation, and homogeneous cluster nucleation from $-\frac{1}{2} \mathbf{C}^\top \mathbf{R}_2 \mathbf{C}$ — assembled into the linear form $(\text{test}(c_I), \dot{c}_I) dV$ and then *projected* onto the trial space by solving the consistent mass matrix $\mathbf{M} = (\text{test}(c_I), c_I) dV$ with conjugate gradients to 10^{-4} (`SpatialODESolver`). The stored rate is $\mathbf{M}^{-1} \mathbf{b}$.

Table 15: Immobile-species integration across the three code states. The upstream column is MoDELib-fullCD, read from a checkout, not inferred.

	MoDELib-fullCD (upstream)	MoDELib3 (standalone route)	ZrMicro coupled march (this work)
iSize (immobile dofs/node)	8	8	—
Rate discretization	Galerkin: consistent-mass L^2 projection, CG to 10^{-4}	nodal collocation, no mass matrix	pointwise, per node
Time update	explicit Euler on the projected rate, then sequential implicit loss factors	one fused semi-implicit update	CVODE BDF
Order	1	1	up to 5
Step control	one step of dtMax	fixed, substeps/dpa	20 adaptive
Error control	none	none	rtol= 10^{-6}
Loop sink on the mobile	FirstOrderReaction	ImmobileSinks (re-named)	frozen $\mathbf{C}_M^*$
Continuum $\rightarrow$ discrete loops	initializeDiscreteClusters	absent	n/a

2. **Explicit Euler.** `updateImmobileClusters(dt)` does $\mathbf{c}_I \leftarrow \mathbf{c}_I + \dot{\mathbf{c}}_I dt$ with $dt = dtMax$ — one step per DD step, no sub-cycling.

3. **Implicit loss corrections, applied in sequence.** `applyImmobilePopulationCorrections` then divides the result by $(1 + dt \lambda)$ once per loss channel, in a fixed order: thermal annealing (with the content debited by $n_{nuc}(N_{old} - N_{new})$), content removal, then loop-loop and loop-network coalescence with their Avrami factors. A clamp enforces $N \geq 0$ and a minimum loop content $c \geq n_{min} \Omega N$.

MoDELib3 keeps the physics of stages 1 and 3 and changes how both are discretized. The rate is evaluated **at nodes** rather than projected, so the mass matrix and its CG solve disappear; and the explicit step and the loss divisions are **fused into a single semi-implicit formula**, $n^{new} = (n + dt \dot{n}_{nuc}) / (1 + dt \lambda_{tot})$, sub-cycled 20 times per dose step.

Neither choice is strictly better, and the trade is worth stating. The Galerkin projection is the more accurate representation of a smooth field, because it respects the element shape functions instead of sampling them at nodes; but it couples neighbouring nodes, and on a sharp gradient it overshoots. fullCD says so in its own comment — the corrected sink value exists “to adjust for negative values at the barycenter of the quadratic elements where we have a sharp gradient of sink values between two end nodes”. Nodal collocation cannot produce that artifact, at the cost of ignoring the sub-element variation entirely. Given that the $1 \mu m$ case has a boundary layer spanning twelve orders of magnitude in C_v (Table 21), sharp gradients are the normal condition here rather than the exception, and MoDELib3’s choice is defensible on those grounds — though it was not, so far as the change log records, made for that reason.

The sequencing also differs in a way that matters for large dt . fullCD applies its loss factors *one after another*, so with three active channels the result is $\prod_k (1 + dt \lambda_k)^{-1}$, whereas MoDELib3’s fused form gives $(1 + dt \sum_k \lambda_k)^{-1}$. The two agree to $O(dt^2)$ and diverge when any $dt \lambda_k \gtrsim 1$, which — given the measured $dt \lambda \approx 46\,000$ of Section 7 — is the operating regime, not a corner case. Both remain unconditionally positive, which is the property that matters most here; neither is a controlled approximation at that step size.

8.2 How MoDELib3’s variant reached its present form

MoDELib3 re-derived its update rather than porting fullCD’s, and in doing so it re-encountered the stiffness that fullCD’s loss factors already handle. It began fully explicit and reached the semi-implicit update through four documented corrections, recorded in `MoDELib3/ZR3D_GHONIEM_CHANGES.md`:

Table 16: How the standalone immobile scheme reached its present form. Each stiffness failure was fixed by moving one more linear loss into the implicit denominator.

Issue	Symptom	Cause	Fix
7	$\langle c \rangle$ loops pinned on the positivity floor while $\langle a \rangle$ loops nucleated normally	τ_{vL} is $35\times$ shorter than the substep, so explicit Euler gave $n(1 - 35) < 0$, clamped every iteration	Integrate the linear losses implicitly; returns $n^\star = \dot{n}_{\text{nuc}}\tau_{vL}$ exactly as $dt/\tau \rightarrow \infty$
8	Thermal emission could not be made implicit	It was written as an absolute rate, not proportional to the stored content	Recast as a first-order coefficient (divide by c) so it can enter the implicit update
14	$\langle a \rangle$ mean size 5042 at 1 dpa, then exactly $300.0 = n_{\text{nuc}}$ at 2 dpa — content collapsing $11\times$	The same stiffness failure in the coalescence channel left explicit: $\nu_{LN}\phi_{LN}dt_{\text{sub}} \approx 3.6$	Eq. (97) removes n and c proportionally, so both are first-order coefficients; moved implicit. $\langle a \rangle$ density went from $3\text{--}4\times$ the 0-D to within $\sim 30\%$
15	$\langle c \rangle$ content $350\times$ below the 0-D, mean size pinned at the nucleation size	Dissolution removed content proportional to the stored content	Release the birth content $n_{\text{nuc}}n/\tau_{vL}$ instead

Issue 7 is worth singling out because its diagnosis was made the right way: the authors confirmed it was integration and not physics by checking the analytic balance $n^\star = \dot{n}_{\text{nuc}}\tau_{vL} = 1.79 \times 10^{-8}$ against the 0-D $C_{vL} + C_{avL} = 1.78 \times 10^{-8}$, then verified the fix reproduced 1.79161×10^{-8} .

8.3 The coupled march scheme: outward from the 0-D solver

The march scheme has the opposite history. It did not have to discover stiffness, because it inherited a stiff integrator: ZrMicro’s CVODE BDF solver predates the coupling entirely, built for the standalone 0-D 19-equation model. The coupling was then added in three steps:

1. **freeze_mobile** — the operator split. The four mobile derivatives are zeroed *last*, after every reaction term has already read them as parameters, so the same right-hand side serves as the local immobile micro-model with an externally supplied $\mathbf{C}_M^\star$.
2. **Pointwise dispatch** — because the split makes the slow march decoupled between points, each quadrature point becomes one case of the existing OpenMP `-batch_file` mode.
3. **This work** — accumulators moved to CVODES quadrature ($19 \rightarrow 9$), the exact AD Jacobian, and per-thread workspace reuse.

8.4 Three routes to the same conclusion

All three codes end up treating the immobile loss channels implicitly, and all three are right to, but they arrive from three different directions — and the agreement is stronger evidence than any one of them alone.

- **fullCD** put the losses in implicit denominators *by construction*: annealing, content removal and both coalescence channels are applied as $(1 + dt \lambda)^{-1}$ factors from the outset, while the source terms stay explicit. It is a deliberate IMEX split.
- **MoDELib3** re-derived the update, started fully explicit, and found the stiffness *empirically* — twice, as loop populations collapsing onto the positivity floor (issues 7 and 14) — then fixed it term by term until it had reconstructed the same partition fullCD already had.
- **The 0-D side** never exposed the problem at all, because BDF absorbed it silently from the start; it took the spectrum of Table 11 to put a number on what was being absorbed ($46\,000\times$ the forward-Euler limit).

That MoDELib3 rediscovered fullCD’s partition by symptom, one channel at a time, is the clearest single argument that the partition is forced by the physics and not a modelling preference. It is also an argument for reading the upstream: the two failures documented as issues 7 and 14 were both already answered by `applyImmobilePopulationCorrections`.

That difference in route leaves a difference in robustness, and it is the practical point of this section. MoDELib3’s scheme is unconditionally stable on the losses that *have been* moved into `lossN/lossC`. It offers no protection for one that has not. A newly added immobile loss term will be stable only if it is first recast as a first-order coefficient and placed in the denominator; written as an explicit rate it will reproduce issue 7 or issue 14, and — with no local error estimate and no step rejection — the only symptom will again be a species quietly pinned on the floor. CVODE would have flagged the same term through step rejection and error-test failures without anyone having to notice a collapsed population.

Sourcing. The upstream statements in this section were read directly from a checkout of `mlm335/MoDELib-fullCD` at commit `62ac34f` (`src/ClusterDynamics/`, `include/ClusterDynamics/`, `Library/Materials/`). MoDELib3/ itself carries no independent git history, having been flattened from a submodule — the commits present are DisloCluster’s own — so the fork point cannot be recovered from the repository and the correspondence above is established by reading the two sources side by side rather than from history.

One earlier piece of evidence should be retired rather than repeated. `MoDELib3/Library/Materials/Zr4.txt` does contain zero immobile keys, and that fact is correct; the *inference* drawn from it — that the upstream standalone route therefore could not produce an $\langle a \rangle / \langle c \rangle$ microstructure at all — was wrong. `Zr4.txt` is simply an older material file that happens to be carried in this checkout. Upstream, `Zr4_Fitted.txt` and `Zr4_BMD19_nuc.txt` each carry the full sixteen-key immobile set (`immobileSpeciesVector`, `loopCascadeFraction`, `loopNucDefects`, `loopContentTau_SI`, `loopAnnealTau0_SI`, `loopCoal*`, `n_min`, `dislocationSinks_SI`). The absence of those keys from one file in this repository says nothing about the upstream code.

9 The three recommendations, implemented and measured

The three items at the head of the earlier recommendation list have been built and measured on the real field: $q = 24\,115$ CD nodes of the $1\,\mu\text{m}$ cube, one 1 dpa substep, seeded from the shared 0.1 dpa state with the mobile field taken from the 3-D solve.

9.1 Recommendation 1: how the accumulators are carried

The choice is now explicit, `-acc_mode`:

Mode	Treatment
0	In the implicit state <i>and</i> in the error test — the legacy layout.
1	CVODES quadrature variables. Smallest Newton block, but the quadrature right-hand side needs its own core evaluation at the accepted step point, and the memo meant to reuse the residual’s evaluation essentially never hits (Section 5.5).
2	In the implicit state, with a per-component <code>atol</code> large enough that their error weights vanish (<code>CVodeSVtolerances</code>). Bigger Newton block, but <i>zero</i> extra core evaluations.

Table 17: Accumulator handling, $q = 24\,115$, one 1 dpa substep, AD Jacobian. Errors are against mode 0.

Mode	n	wall [s]	cpu [s]	steps	core evals	n_{LU}	LU flops
0 in state, error-controlled	19	4.193	74.01	3 346 941	4 184 924	961 510	2.20×10^9
1 CVODES quadrature	9	3.402	49.84	2 508 983	5 838 440	584 867	1.42×10^8
2 in state, atol relaxed	15	3.314	49.06	2 397 987	3 077 945	507 161	5.71×10^8

The prediction of Section 5.5 is confirmed, and quantified. Mode 1 has the smallest Newton block by a wide margin — an order of magnitude fewer LU flops than mode 0 — yet the *largest* core-evaluation count of the three, because the quadrature adds roughly one full sweep per step. Mode 2 gives that LU saving back and wins where the cost actually is: **26% fewer core evaluations than the legacy layout and 47% fewer than the quadrature**, with the fewest steps of the three. Modes 1 and 2 agree with each other to 1.7×10^{-5} against mode 0, which is the expected consequence of removing the accumulators from the error test, not a disagreement between them.

Mode 2 is now the coupling default. Mode 0 remains the default for standalone 0-D runs, which stay bit-identical.

9.2 Recommendation 2: the base+delta case protocol

Every batch line used to repeat the full parameter set. A line beginning `@BASE` now carries the shared defaults once and each case line only its overrides; a file without a `@BASE` line still parses exactly as before, and the solver reports on `stderr` which protocol it used so a benchmark cannot silently measure the wrong one.

Table 18: Case-file protocol, $q = 24\,115$. The two rows of each pair differ only in how the same cases are serialized. The upper block is the seed step, where the immobile state is still uniform and only the four mobile values differ; the lower block is the realistic mid-march state, where all thirteen trajectory components differ per node.

State	Protocol	file [MB]	tokens/case	wall [s]
uniform seed	full-line	35.39	88	5.410
	base+delta	1.58	4	3.407
realistic mid-march	full-line	36.75	89	6.911
	base+delta	7.13	13	5.369

The honest figure is the lower block: $1.29\times$ end to end, with the case file shrinking $5\times$. The upper block would read $1.59\times$, but only because a uniform seed leaves just four values varying per case; quoting it as the general result would overstate the gain, since after the first substep the immobile state varies node to node and thirteen values differ.

9.3 Recommendation 3: deduplicating points before dispatch

Neighbouring nodes in a smooth field integrate nearly identical initial-value problems. `run_immobile_step` now optionally groups points whose trajectory-determining components (the four frozen mobile values, the eight immobile components and ρ_N — not the accumulators, which reset every substep) agree to a relative tolerance, integrates one representative per group and scatters the endpoint back.

Table 19: Deduplication, $q = 24\,115$, one 1 dpa substep. Errors are against no deduplication.

dedup_rtol	integrations	factor	wall [s]	max rel. error
off	24 115	1.00	8.964	—
10^{-10}	17 582	1.37	6.042	0 (exact)
10^{-8}	17 582	1.37	5.983	0 (exact)
10^{-6}	17 582	1.37	6.610	0 (exact)
10^{-4}	15 753	1.53	5.683	9.9×10^{-5}
10^{-2}	6 368	3.79	2.301	9.7×10^{-3}

This is well short of the 10^2 – 10^3 collapse the earlier note speculated about, and the reason is worth recording. Tolerances from 10^{-10} to 10^{-6} all find the *same* 17 582 distinct states: those 6 533 merges are **exact** duplicates, and the error is not merely small but identically zero. They are the Dirichlet-pinned boundary nodes, where the mobile field is clamped to the same value. Away from the boundary the field varies smoothly and continuously across eight decades, so genuinely distinct states stay distinct at any tolerance tight enough to be free.

Buying more requires accepting error: 10^{-4} gives $1.53\times$ for 10^{-4} relative error, and 10^{-2} gives $3.79\times$ for about 1%. At an integrator `rtol` of 10^{-6} the defensible setting is 10^{-8} , which is what the coupling uses — $1.37\times$ **fewer integrations at zero error**. The speculation that this would dominate everything else was wrong for this field, and the measurement says so.

9.4 Combined effect

Taken together on the coupling path — mode 2, base+delta, deduplication at 10^{-8} , on top of the workspace reuse and AD Jacobian of the earlier sections — the per-substep cost at $q = 24\,115$ falls from the 8.96 s of the unduplicated legacy-protocol path to 5.98 s, while the core-evaluation count drops 26% and the roundoff-warning pathology stays cured.

10 The 1 μm reference through both routes

Both routes were run on the 1 μm single crystal from *one* shared microstructure — the 0-D state at 0.1 dpa written uniformly onto all 24 115 CD nodes — and advanced to 20.1 dpa in 5 dpa intervals.

Both run the *same* operator split and the same fast step; they differ in what advances the immobile population.

Route	Immobile advance
standalone	MoDELib3's own nodal semi-implicit scheme, 20 substeps per 1 dpa step, run by DDomp.
coupled	CVODE BDF at every node, <code>acc_mode=2</code> , exact AD Jacobian, workspace reuse, base+delta protocol, deduplication at 10^{-8} . The mobile field is re-solved by MoDELib3 every <code>FEM_EVERY</code> substeps and frozen in between.

10.1 The fast step belongs inside the loop

The first version of this comparison did not perform an operator split at all, and the defect is worth stating plainly because it invalidated part of the earlier numbers.

MoDELib3's `ClusterDynamicsFEM::solve()` *is* the split:

```

solveMobileClusters();      // FAST -- steady C_M*(x). No time derivative
                             // anywhere; the immobile population enters only
                             // through ImmobileSinks, evaluated at every
                             // quadrature point from the LOCAL state.
solveImmobileClusters();    // SLOW -- dt update of the nodal n_I, c_I

```

The coupled route was meant to keep the first call and replace the second. What it actually did was read the mobile field out of the *standalone run's* recorded snapshots. Under that arrangement $\mathbf{C}_M$ never responds to the immobile state the march is building: the coupling is one-way, and the march has no mechanism to relax a seed that is not already at quasi-steady state. Worse, the nearest recorded snapshot to the 0.1 dpa seed sits at 1.1 dpa, outside the 0.75 dpa acceptance window, so the entire first interval — a quarter of the march — ran on the raw uniform seed field.

The fix is a new DD.txt scalar, `useImmobileSolver` (default 1, so every pre-existing case is untouched). With `useImmobileSolver=0` a DDomp run performs the fast step and passes the immobile field straight through. The march then calls it as a subroutine: write the current per-node immobile state into the `ev1` CD block, run one DDomp step to obtain $\mathbf{C}_M^*(x)$ *for that state*, freeze it, and integrate the immobile ODEs at every node. Because `runSingleStep` writes output *before* incrementing `runID`, a run with `startAtTimeStep=0` and `Nsteps=1` reads `ev1_0.txt` and overwrites the same file, which makes the exchange an in-place round trip. `py_utils/modelib_qssa.py` drives it and asserts on DDomp's immobile solver `SKIPPED` line, so a binary predating the flag fails loudly rather than advancing the immobile field twice.

10.2 A non-terminating mobile solve, and its cause

Closing the loop exposed a defect that the standalone route had never triggered. Asked to solve the mobile field against an immobile state supplied by the march, `solveMobileClusters` does not converge: `cError` falls from 5.8×10^5 to 1.3×10^{-4} in nine iterations and then oscillates around 5×10^{-5} indefinitely. The loop is

```
while (cError > cTol)
```

with `cTol =  $10^{-5}$` , *no iteration cap and no stagnation test* — inherited verbatim, since MoDELib2-NNL, MoDELib-fullCD and MoDELib3 all carry the identical loop. So DDomp never returns.

Cause. The one structural difference from upstream is the cause. `fullCD` has *no mobile clamp at all*; MoDELib3 inserted `clampMobileClusters()` between the Newton update and the error test, which makes the iteration a *projected* Newton method.

The obvious explanation — that the active set oscillates — is wrong, and an instrumented run refutes it: the set freezes at 7090 of 96460 dofs by iteration 3 and reports zero entering and zero leaving on every iteration after. What actually fails is that the Newton matrix

dmBWF+bWF_R1+bWF_R2+bWF_RI is the *unconstrained* Jacobian and does not know any dof is pinned. Each iteration it computes an increment for every dof, including moves for the pinned ones; the projection discards exactly those moves, but the increment for the *free* dofs was computed on the assumption that the pinned ones would move. The free dofs therefore respond, every iteration, to a coupling that is immediately cancelled, and that inconsistency does not decay. Solve-then-project is not a consistent method for a constrained system; the consistent form is an active-set method, in which the pinned rows are eliminated and the reduced system solved.

The intent is visible in the routine’s own comment — the clamp mirrors ZrMicro, “which applies $y = \max(y, C_{\text{floor}})$ before every *rate evaluation*”. In ZrMicro the floor belongs to the rate evaluation. Here it mutates the state inside the iteration, and only that breaks convergence.

Table 20: Mobile solve on an identical marched immobile state, 24 115 CD nodes. The last column is the maximum relative error in C_i against the converged field, over the 22 344 nodes above the floor.

Configuration	iters	wall [s]	converged	cError	max err. C_i
clamp in loop (as shipped)	30*	392	no	6.06×10^{-5}	53.6%
clamp deferred	5	112	yes	6.83×10^{-7}	—
clamp deferred, cTo1= 10^{-8}	6	120	yes	2.26×10^{-15}	0.0
damped $w = 0.5$	30*	400	no	9.42×10^{-5}	4.0%
damped $w = 0.7$	20	283	yes	8.21×10^{-6}	0.19%
error test excludes floored dofs	40*	492	no	4.67×10^{-5}	21.1%
cTo1 relaxed to 10^{-4}	19	258	“yes”	9.22×10^{-5}	49.9%

* stopped at the iteration cap.

Every remedy, measured on one saved state. Deferring the clamp restores **quadratic** convergence — $2.95 \times 10^4 \rightarrow 1.23 \times 10^3 \rightarrow 6.77 \times 10^1 \rightarrow 5.73 \times 10^{-2} \rightarrow 6.83 \times 10^{-7} \rightarrow 2.26 \times 10^{-15}$, each error the square of the last — and is the fastest option by a factor of 2.5.

Three of the alternatives are instructive failures. Changing the error metric is *bit-identical* to the baseline, all sixteen digits, iteration for iteration: a floored dof has $c^{\text{new}} = c^{\text{old}}$ and already contributes nothing to the step norm, so the free dofs are genuinely oscillating. Under-relaxation does converge, but only linearly, at roughly a factor of two per iteration. And relaxing cTo1 is actively dangerous: it reports success at iteration 19 while accepting a field that is 49.9% wrong in C_i at the worst node.

Verification. The chosen fix was checked four ways rather than assumed from the fact that it terminates. Convergence is quadratic; the fields at cTo1 = 10^{-5} and 10^{-8} are *bit-identical*, so five iterations already reach machine precision; the independent damped route reaches the same field to within 0.19%, with the means agreeing to five figures, so the clamp was corrupting the iteration and not the answer; and the result is physical — zero negative entries and exactly the same 7090 floored dofs as the clamped runs, because the floor is still applied, once, after the loop.

The stalled iterate is not merely unrecognized as converged. Its means agree with the converged field to four figures, but two nodes exceed 1% and one reaches 53.6% in C_i . A handful of nodes at the clamp boundary drives the whole failure, and because the error test is a maximum over nodes, those few alone prevent it from ever settling.

What was changed. Five optional DD.txt scalars (mobileSolverTolerance, mobileSolverMaxIterations, mobileSolverRelaxation, mobileSolverErrorMode, mobileSolverClampInLoop), all defaulting to the historical behaviour except the iteration cap, which now retains the *best* iterate and so converts a hang into a bounded, reproducible result. The coupled fast solve sets mobileSolverClampInLoop=0 in its own simulation directory and warns if any solve returns capped rather than converged. The

standalone route’s defaults are deliberately left unchanged: making the deferral global is very likely correct, but it alters a path that currently runs to completion, and that warrants a full standalone verification first.

10.3 Does the split repair a seed that is not quasi-steady?

This is the question the arrangement exists to answer, and it can be asked directly: solve the fast step once against the seed’s own immobile state, before any immobile marching, and compare with what was seeded.

Table 21: The 0.1 dpa 0-D seed against the quasi-steady field solved for that same immobile state. 24 115 CD nodes, 573 K, 10^{-7} dpa/s. Reproduced to the digit on an independent invocation.

Species	seeded (uniform)	QSSA mean	seed / QSSA	QSSA min	QSSA max
C_v	1.134×10^{-5}	1.643×10^{-8}	690	1.0×10^{-20}	6.15×10^{-8}
C_i	1.645×10^{-10}	1.754×10^{-12}	94	4.1×10^{-27}	6.77×10^{-12}
C_{2i}	1.113×10^{-12}	8.966×10^{-15}	124	3.4×10^{-53}	3.46×10^{-14}
C_{3i}	4.455×10^{-13}	5.977×10^{-15}	75	1.3×10^{-70}	2.31×10^{-14}

The seed was very far from quasi-steady, and the fast step supplies the whole difference. Two corrections are happening at once, and they are worth separating:

- **Magnitude.** The seeded C_v is $690\times$ the quasi-steady mean. The 0-D reaches its own steady balance against a homogenized sink strength; the 3-D domain has, in addition, six Dirichlet faces holding the mobile species at thermal equilibrium and draining the interior.
- **Structure.** The seed is uniform by construction, so its spatial contrast is exactly 1. The relaxed C_v field runs from 10^{-20} at the faces to 6.15×10^{-8} in the interior — twelve orders of magnitude. The whole boundary layer is absent from the seed and is created by the fast solve. No amount of immobile marching could have produced it, because the immobile equations carry no spatial derivative at all.

The C_i minimum of 4.1×10^{-27} is the imposed boundary value $\exp(-3.0 \text{ eV}/kT)$, deliberately *not* floored at 10^{-20} : flooring it would raise the boundary concentration by seven orders and quietly disable the grain boundary as an interstitial sink, which is the one piece of physics the 3-D model adds over the 0-D.

So the answer is yes. The split reaches quasi-steady state on its own, and it does not need the 0-D seed to be right — which is fortunate, because a 0-D seed can never be right about a boundary layer it does not represent.

10.4 What this does and does not isolate

With the fast step closed, both routes now solve the same fast problem with the same code, each against its own immobile state. Two differences remain, and neither is the integrator alone:

1. **Different immobile equations.** The coupled route integrates ZrMicro’s immobile right-hand side; the standalone integrates MoDELib3’s. These are two models, not two integrations of one model.
2. **Different integrators**, as characterized in Section 7, and a different splitting cadence: FEM_EVERY substeps between fast solves against the standalone’s one per internal substep.

Attributing any number below to the integrator alone would therefore still be wrong. Section 10.6 separates the contributions.

10.5 Speed

Table 22: Wall clock for the full 0.1–20.1 dpa schedule, 24 cores.

Route	Integrator	wall [s]	per dpa [s]
standalone	MoDELib3 semi-implicit, 20 substeps/dpa, nodal	2409	120.5
coupled ($\Delta\gamma = 5$ dpa)	CVODE BDF, <code>acc_mode=2</code> , AD Jacobian	424	21.2
coupled ($\Delta\gamma = 1$ dpa)	as above, mobile refreshed $5\times$ more often	526	26.3

The coupled march is $5.7\times$ faster than the standalone solve at equal dose span. That is not a like-for-like statement about integrators: the standalone also solves the mobile FEM boundary-value problem at every step, which the coupled route reads from a file rather than computing. It is a statement about the two routes as a user runs them.

10.6 Attribution: model, splitting, or integrator?

The decisive measurement is to place both routes against the plain 0-D model integrated over the same dose range.

Table 23: Domain-mean $\langle a \rangle$ loop number density (atom fraction). “0-D alone” is ZrMicro integrated to the same dose with no spatial coupling at all.

Dose [dpa]	0-D alone	coupled	standalone	coupled / 0-D	standalone / 0-D
5.1	1.03×10^{-2}	2.61×10^{-3}	1.43×10^{-4}	0.255	0.0140
10.1	1.84×10^{-2}	8.79×10^{-3}	1.53×10^{-4}	0.478	0.0083
15.1	2.14×10^{-2}	1.56×10^{-2}	1.62×10^{-4}	0.728	0.0076
20.1	2.32×10^{-2}	2.23×10^{-2}	1.72×10^{-4}	0.961	0.0074

The coupled route **converges onto the plain 0-D answer** as dose accumulates ($0.255 \rightarrow 0.961$): the spatially resolved mobile field delays the 0-D result but does not change where it goes. The standalone sits about $130\times$ below the 0-D throughout.

Refining the splitting interval settles the remaining ambiguity. Halving is not enough to be conclusive, so the interval was cut by five, holding the total substep count fixed:

	$\Delta\gamma = 5$ dpa	$\Delta\gamma = 1$ dpa	change
coupled $\langle a \rangle$ at 20.1 dpa	2.23×10^{-2}	2.77×10^{-2}	+24%
ratio to standalone	130	161	—

A five-fold refinement of the splitting moves the answer by 24%, and moves it *away* from the standalone rather than toward it. So:

- the **model difference is** $\sim 130\times$ and dominates everything else by two orders of magnitude;
- the **splitting error is** $\sim 20\text{--}25\%$ at $\Delta\gamma = 5$ dpa, and is not what separates the routes;
- the **integrator difference is bounded well below both**, consistent with Section 7.

The $\langle c \rangle$ family shows the mirror image: the standalone holds 1.78×10^{-8} while the coupled route sits at 5.50×10^{-10} , a factor 32 the other way, flat in dose in both routes.

10.7 A discrepancy with the calibration record

This conflicts with what `ZR3D_GHONIEM_CHANGES.md` records. Its issue 14 states that after moving coalescence into the implicit update, “ $\langle a \rangle$ loop density went from 3–4 $\times$ the 0-D to within $\sim 30\%$ ”. Here MoDELib3 sits at 0.7–1.4% of the 0-D — roughly 100 $\times$ *below*, not within 30%.

One difference from the calibration conditions is visible in the run itself and should be checked first. The calibration used the tutorial’s *generated* microstructure; this run seeds MoDELib with the 0-D state at 0.1 dpa, for which $N_a = 4.94 \times 10^{-4}$. MoDELib immediately coalesces that down to 1.43×10^{-4} by 5.1 dpa and then holds it essentially flat (1.72×10^{-4} at 20.1 dpa) while the 0-D grows a hundredfold. Whether that is the seeded start putting MoDELib outside its calibrated regime, or a genuine regression in the calibration, is not resolved here. It is flagged rather than explained, and no physics conclusion should be drawn from these fields until it is.

10.8 Output

The run writes the reference layout under `output/<stamp>_<hash>_coupled_vs_standalone/`: `0d/`, `3d/standalone/` and `3d/coupled/`, `gb/` for both routes, `tables/` with the comparison in Markdown, CSV and JSON, and `ev1_coupled/` holding the marched CD fields as complete, restartable MoDELib configurations. `run_coupled_vs_standalone.py` drives it, and Section 14 of `coupled_0d_3d_ZrMicro.ipynb` calls that driver.

11 Summary and recommended next steps

Table 24: Summary of outcomes against intent.

Change	Target metric	Outcome
AD Jacobian	residual evaluations	–33%; wall neutral
9×9 reduction	LU flops	–92%; steps –18%; wall –4%
	step-size pathology	2048 roundoff warnings $\rightarrow$ 0
Workspace reuse	solver setup cost	march wall $\times 1.67$ at $q = 1024$
Field handoff	information exchanged	4 scalars $\rightarrow$ 289 380 values
Full 1–26 dpa march	same-work solver speed	1.95 $\times$ (Table 10)

Two findings are worth carrying forward independently of the changes themselves. First, the accumulators were quietly setting the initial step of every coupling substep, and removing them from the error test cures a roundoff-limited first step that had been occurring 2048 times per march (Section 5.4). Second, the two codes integrate the same immobile equations to very different error levels — first order with no error control on the 3-D side, adaptive to 10^{-6} on the 0-D side — which is a systematic 0-D/3-D discrepancy distinct from the three physics non-correspondences on record (Section 7).

Items 1–3 of the original list have been implemented and measured (Section 9); what follows is what remains, ordered by expected return.

1. **Drive MoDELib in process.** This is now the binding constraint. Item 1 of the previous version of this list — put the FEM mobile solve inside the march loop — has been done (Section 10.1), and doing it exposed the cost: every fast solve is a separate DDomp invocation, so the mesh load, the FE assembly and the factorization of the diffusion operator are repeated on each of them, at 174s per call, against a standalone run that amortizes all of it once over twenty steps. `pyMoDELib` removes the restart, and with it the dominant cost of a short splitting interval. Until then `FEM EVERY` is as much a wall-clock knob as a numerical one.

2. **Guard the standalone immobile solver against the next stiff term.** Section 8 shows the same failure was found twice, by symptom, and there is still no mechanism that would catch a third. Any new loss channel added to `solveImmobileClusters` must be recast as a first-order coefficient and placed in `lossN/lossC`; a cheap standing check is to run one case at `nSub` and at `10 nSub` and assert the answers agree, which turns a silent floor collapse into a visible regression.
3. **Revisit GPU offload only after the above.** With the core-evaluation count now the dominant cost and the LU already irrelevant at $n = 15$, the case for batched GPU integration rests entirely on how large q becomes in a live coupling — which is not yet known.

Files. `cpp_utils/dual.h`, `cpp_utils/rate_equations_core.h`, `cpp_utils/rate_equations.{h,cpp}`, `cpp_utils/jac_check.cpp`, `cpp_utils/parameters.h`, `code/solver.cpp`, `py_utils/cpp_bridge.py`, `py_utils/modelib_coupling.py`, `py_utils/modelib_field.py` (new), `py_utils/run_coupled_vs_standalone.py` (new), `code/coupled_0d_3d_ZrMicro.ipynb`. The build now links `SUNDIALS::cvodes` rather than `SUNDIALS::cvsode`, because `CVodeQuadInit` and its companions exist only in `CVODES`; the header `<cvodes/cvodes.h>` was already being included.